K-NEAREST NEIGHBORS (KNN)

1. PENGERTIAN

K-Nearest Neighbors (KNN) adalah algoritma Supervised Learning yang digunakan untuk melakukan klasifikasi maupun regresi dengan cara mencari K data training yang memiliki jarak paling dekat terhadap data baru.

- Prinsip utama KNN adalah: Data yang memiliki karakteristik mirip cenderung memiliki karakteristik atau label yang sama.

KNN menentukan kemiripan berdasarkan perhitungan jarak matematis antara data baru dengan seluruh data training.

2. KONSEP DASAR DAN ANALOGI SEDERHANA

Bayangkan kamu merupakan siswa baru di sebuah sekolah. Kamu ingin mengetahui apakah seseorang memiliki minat terhadap Matematika atau Olahraga. Kamu belum mengenal orang tersebut. Kemudian, kamu mencari lima orang yang memiliki karakteristik paling mirip dengannya.

Hasilnya:

- 4 orang menyukai Matematika.
- 1 orang menyukai Olahraga.

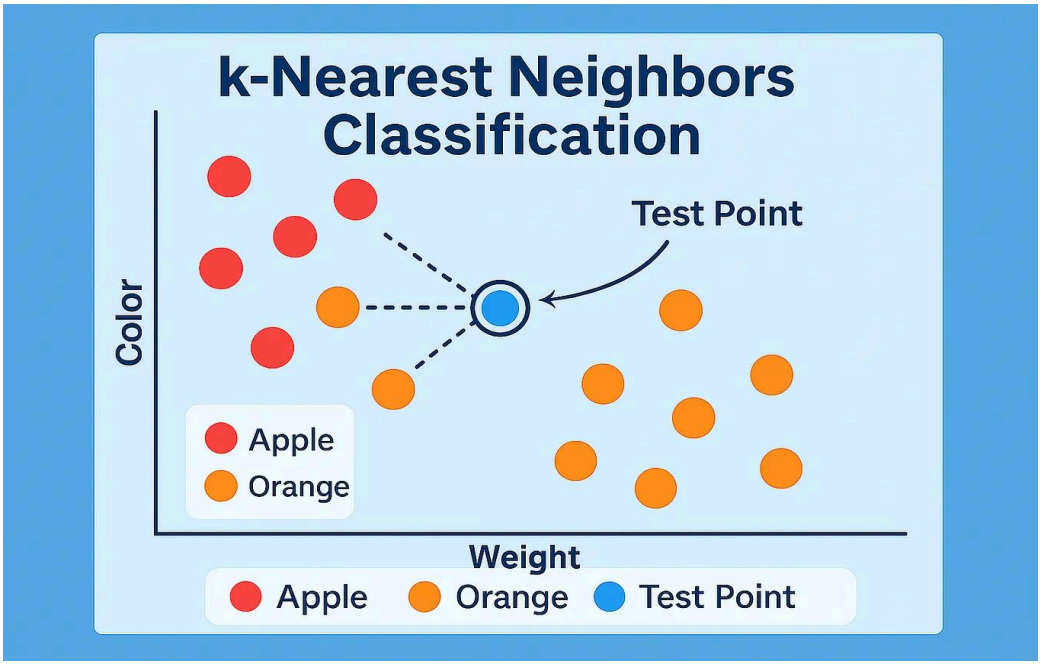
Kamu kemudian memperkirakan bahwa orang tersebut kemungkinan besar menyukai Matematika.

Dalam algoritma:

- Orang baru = Data yang ingin diprediksi.
- Lima orang = K = 5.
- Kemiripan = Jarak matematis.
- Hasil suara terbanyak = Prediksi.

Inti konsep KNN adalah mencari tetangga terdekat, kemudian menggunakan informasi dari tetangga tersebut untuk menentukan prediksi.

3. VISUALISASI CARA KERJA KNN



4. TAHAPAN ALGORITMA KNN

Tahap 1 - Menyiapkan Dataset

Tahap 2 - Memasukkan Data Baru

Tahap 3 - Menghitung Jarak

Hitung jarak antara data baru dengan seluruh data training.

Tahap 4 - Mengurutkan Jarak

Urutkan jarak dari yang terkecil hingga terbesar.

Tahap 5 - Memilih K Tetangga

Ambil K data dengan jarak terkecil.

Tahap 6 - Majority Voting

Hitung jumlah kemunculan masing-masing kelas.

Tahap 7 - Menentukan Prediksi

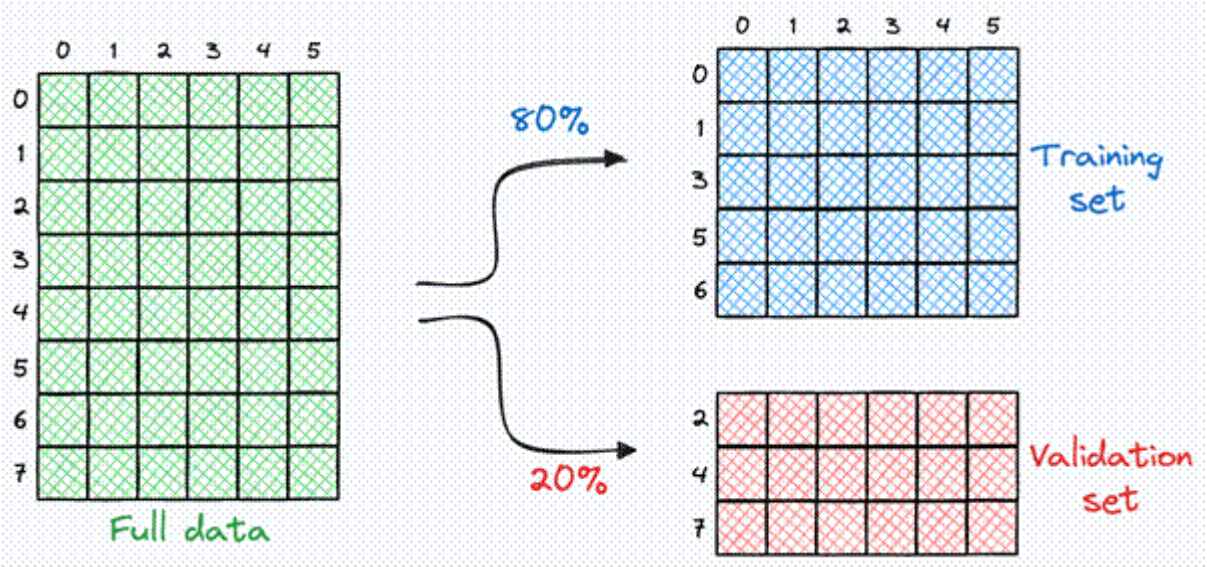
Kelas dengan jumlah suara terbanyak menjadi hasil prediksi.

- **Langkah - Langkah Membuat Algoritma KNN**

1. **Menentukan Variabel Independent (Fitur) & Dependent (Target)**

Fitur adalah variabel yang digunakan untuk menggambarkan data dalam suatu model. Sedangkan **target** adalah variabel yang ingin diprediksi, hasil atau label yang ingin kita ramalkan menggunakan fitur-fitur tersebut.

2. **Split Data (Training & Testing)**



Kenapa data perlu di split? Karena digunakan untuk pelatihan dan evaluasi model pembelajaran mesin. Data biasanya dibagi menjadi dua bagian, yaitu:

a. Training Set (Data Pelatihan): Digunakan untuk melatih model. Di sini, model belajar dari data untuk mengenali pola atau hubungan antara fitur (independent variables) dan label (dependent variables).

b.Test Set (Data Pengujian): Digunakan untuk menguji kemampuan model setelah dilatih. Data ini tidak pernah digunakan dalam proses pelatihan, sehingga membantu mengukur seberapa baik model dapat memprediksi data yang belum pernah dilihat sebelumnya.

3. **Menentukan Nilai K (Tetangga Terdekat)**

a. Cross Validation

Cara umum untuk memilih k adalah dengan menggunakan **cross-validation**, dengan menguji model di berbagai nilai k dan memilih yang memberikan performa terbaik pada validation set. Biasanya, nilai k yang lebih besar lebih baik untuk dataset yang lebih besar, tetapi jika terlalu besar, model bisa menjadi terlalu general, kehilangan kemampuan

untuk mendeteksi pola yang lebih kecil dalam data. Selain itu, untuk memilih nilai k sebaiknya angka ganjil untuk menghindari kemungkinan terjadi seri saat voting, di mana jumlah kelas yang dipilih oleh tetangga terdekat terbagi rata (misalnya, 3 tetangga memilih kelas A dan 3 memilih kelas B).

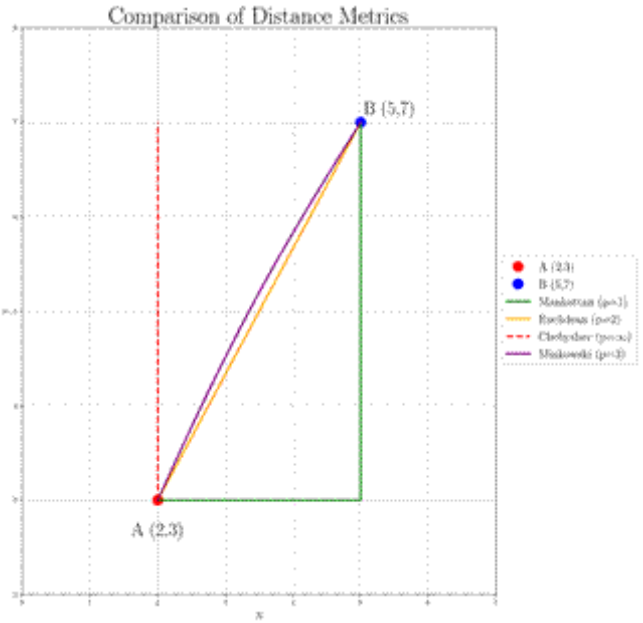
b. Grid Search

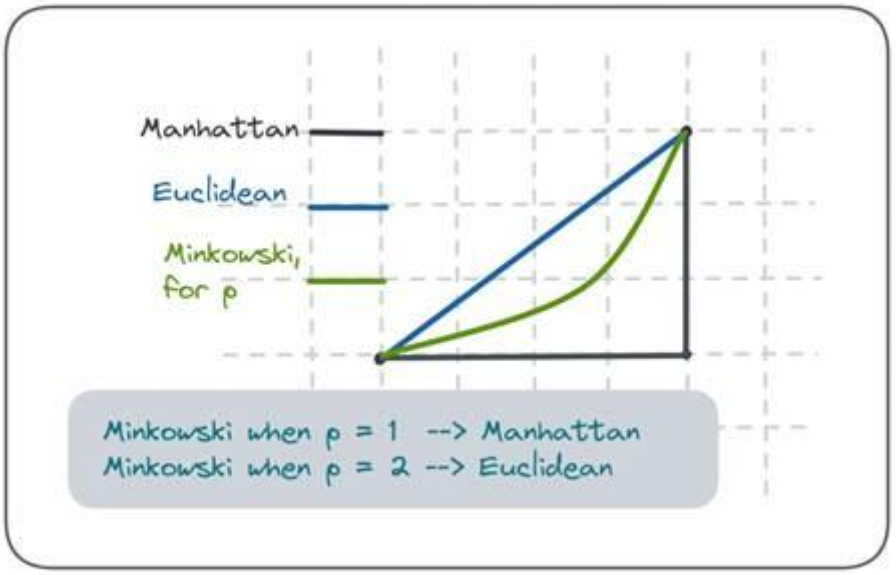
Grid Search adalah teknik untuk mencari **kombinasi parameter terbaik** dengan cara mencoba semua kemungkinan nilai parameter yang telah ditentukan dalam sebuah “grid” (daftar pilihan).

Jika **Cross-Validation** fokus pada mengevaluasi performa model pada 1 nilai parameter,
maka Grid Search fokus pada mencari parameter terbaik dengan cross-validation di setiap kombinasi parameter yang ada.

4. Menghitung Jarak

Untuk setiap data yang ingin diprediksi bisa dihitung jarak antara data tersebut dengan setiap data dalam dataset pelatihan. Biasanya, jarak yang digunakan adalah Euclidean Distance, namun bisa juga menggunakan Manhattan atau jarak lainnya.





1. **Euclidean Distance**

   Euclidean distance, yaitu jarak garis lurus antar dua titik dalam ruang. Ini seperti jarak yang biasa kita bayangkan ketika mengukur secara langsung dari satu titik ke titik lain, sehingga sangat intuitif digunakan pada data numerik yang berskala sama. Di dalam library Scikit-Learn, Euclidean distance ini sebenarnya menjadi default karena metric bawaan KNN adalah Minkowski dengan parameter p=2, yang secara otomatis menjadikannya Euclidean.

   Rumus Euclidean Distance :

$$d(x, x_{train}) = \sqrt{\sum_{j=1}^{n} (x_j - x_{train,j})^2}$$

   xj: Nilai fitur ke-j dari data baru.
   xtrain,j: Nilai fitur ke-j dari sampel data latih.

2. **Manhattan Distance**

   Manhattan distance, yang menghitung jarak berdasarkan penjumlahan selisih absolut tiap dimensi. Ia diibaratkan seperti cara seseorang berjalan di kota berbentuk blok yang hanya bisa lurus ke depan atau ke samping, bukan menyerong. Jarak Manhattan lebih tahan terhadap outlier dan cocok digunakan ketika data memiliki banyak dimensi atau bersifat sparse (banyak nilai nol).

Rumus Manhattan Distance :

$$d(x, x_{train}) = \sum_{j=1}^{n} |x_j - x_{train,j}|$$

xj: Nilai fitur ke-j dari data baru.
xtrain,j: Nilai fitur ke-j dari sampel data latih.

3. **Minkowski Distance**

Minkowski distance, yang merupakan bentuk umum dari Euclidean dan Manhattan. Minkowski menggunakan parameter p sebagai pengatur bentuk pengukuran jarak. Jika nilai p dibuat 1, maka rumus Minkowski otomatis menjadi Manhattan, dan jika p dibuat 2, maka ia menjadi Euclidean. Karena sifatnya fleksibel, Minkowski sering dipakai ketika kita ingin melakukan eksperimen untuk mencari nilai jarak yang paling sesuai dengan karakteristik data, misalnya melalui Grid Search.

Rumus Minkowski Distance :

$$d(x, x_{train}) = \left(\sum_{j=1}^{n} |x_j - x_{train,j}|^p \right)^{\frac{1}{p}}$$

xj: Nilai fitur ke-j dari data baru.
xtrain,j: Nilai fitur ke-j dari sampel data latih.

Contoh Data Dengan Perhitungan Euclidean Distance:

- **Data Training:**
  1. [29, 350]
  2. [51, 430]
  3. [33, 290]
  4. [24, 255]
  5. [40, 410]
  6. [45, 380]

- **Data Baru (Anggap sebuah Data Testing):**

[34, 390]

Data 1

$$dis = \sqrt{(34-29)^2 + (390-350)^2} = 40.31$$

Data 2

$$dis = \sqrt{(34-51)^2 + (390-430)^2} = 43.46$$

Data 3

$$dis = \sqrt{(34-33)^2 + (390-290)^2} = 100.01$$

Data 4

$$dis = \sqrt{(34-24)^2 + (390-255)^2} = 135.37$$

Data 5

$$dis = \sqrt{(34-40)^2 + (390-410)^2} = 20.88$$

Data 6

$$dis = \sqrt{(34-45)^2 + (390-380)^2} = 14.87$$

5. Urutkan Jarak Dari Yang Terkecil Ke Terbesar

Contoh :

Data 6 -> Data 5 -> Data 1 -> Data 2 -> Data 3 -> Data 4

6. Pilih K Tetangga Terdekat

Contoh :

Jika **K = 3**, maka tetangga terdekat kita ambil 3 :
Data 6, Data 5, Data 1

7. Prediksi Dengan Melakukan Voting (Klasifikasi)

a. Majority Voting (Uniform Weight)

Teknik voting yang default, semua tetangga memiliki bobot yang sama namun prediksi dilihat dari jumlah mayoritas label yang muncul.

Rumus :

$$\hat{y} = \arg\max_{c} \sum_{i=1}^{K} \mathbf{1}(y_i = c)$$

K = Jumlah tetangga terdekat
yi = Label kelas tetangga ke-i

b. Distance-Weighted Voting

Menggunakan teknik bobot, pertama hitung bobotnya pastinya jarak terkecil akan lebih besar. Setelah itu dibandingkan dengan bobot lain, untuk mengetahui prediksi labelnya.

Rumus Hitung Bobot:

$$w_i = \frac{1}{d_i + \epsilon}$$

di = jarak tetangga ke-i ke data yang diprediksi
wi = bobot suara tetangga ke-i

Rumus Hitung Bobot Per- kelas:

$$\hat{y} = \arg\max_c \sum_{i=1}^{K} w_i \cdot \mathbf{1}(y_i = c)$$

Contoh :

Tetangga	Jarak d_i	Label	Bobot $w_i = 1/d_i$
Data6	0.10	A	10.00
Data5	0.15	B	6.67
Data1	0.20	B	5.00
Data3	0.30	C	3.33

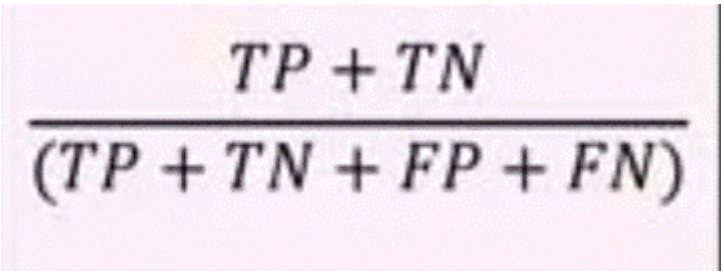
8. Prediksi Pada KNN Regresi

Ambil rata-rata nilai target dari **K tetangga terdekat** untuk membuat prediksi numerik. Misalnya, jika 3 tetangga terdekat memiliki nilai target 5, 7, dan 8, maka prediksi adalah rata-rata dari nilai-nilai tersebut, yaitu (5+7+8)/3 = 6.67.

- Evaluasi Model Klasifikasi

1. **Akurasi**

   Akurasi adalah **Seberapa banyak prediksi model benar secara keseluruhan**. akurasi merupakan metrik yang meringkas performa model klasifikasi secara keseluruhan dengan cara menghitung total prediksi benar dibagi oleh total semua prediksi.
   Metrik akurasi merupakan metrik yang sering digunakan karena mudah dipahami, tetapi metrik ini bisa menyesatkan kita apabila datanya tidak seimbang.
   Kesalahan ini saking seringnya dilakukan oleh pemula sehingga ada nama spesial untuk kesalahan interpretasi ini, disebut "***The Accuracy Paradox***".



$$\frac{TP + TN}{(TP + TN + FP + FN)}$$

2. **Precision**

   Memberikan ringkasan bahwa prediksi kelas positif sebenarnya kelas positif sesungguhnya. Seberapa akurat model **ketika ia mengatakan "positif"**.

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

3. **Recall**

   Memberikan ringkasan bahwa seberapa baik prediksi kelas positif oleh model. Kemampuan model **menemukan seluruh data positif**.

$$Recall = \frac{TP}{TP + FN};$$

4. **F1-score**

metrik perpaduan antara *Precision* & *Recall* dan memberi nilai harmonis antara kedua nilai tersebut (Semakin tinggi, semakin baik).

$$F-score = 2.\frac{precision\ .\ recall}{precision + recall}$$

5. **Confusion Matrix**

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	Negative Predictive Value $\frac{TN}{(TN+FN)}$	Accuracy $\frac{TP+TN}{(TP+TN+FP+FN)}$

		aktual		
		Jahe	Kunyit	Lengkuas
prediksi	Jahe	TP 4 cell 1	2 cell 2 FP	0 cell 3
	Kunyit	1 cell 4	0 cell 5 TN	0 cell 6
	Lengkuas	FN 2 cell 7	5 cell 8	7 cell 9